Predicting Friction in AI-Generated Code: Applying Machine Learning to Anticipate Heavy Review Cycles

Abstract

The increasing adoption of AI coding assistants such as GitHub Copilot, OpenAI Codex, Cursor, Devin, and Claude Code has transformed software development workflows. While these tools accelerate code generation, many AI-generated pull requests (PRs) require extensive human review and multiple revision cycles before acceptance. This project investigates whether machine learning can predict high-friction AI-generated code changes before the review process begins. Using the AIDev-pop dataset containing approximately 33,600 pull requests, we developed classification models to identify PRs likely to require two or more review cycles. Several machine learning algorithms were evaluated, including Logistic Regression, Random Forest, and XGBoost. The final XGBoost model achieved a recall of 80.7% and an F1-score of 61.0% for detecting high-friction PRs. The results demonstrate that predictive analytics can help engineering teams proactively identify problematic code submissions, reduce reviewer workload, and improve developer productivity.

1. Introduction

Recent advancements in generative AI have significantly impacted software engineering practices. AI-powered coding assistants are increasingly used to generate code, automate repetitive tasks, and accelerate software development. Despite their benefits, AI-generated code often requires extensive human intervention to ensure correctness, maintainability, and compliance with project standards.

Code review remains a critical quality assurance mechanism in software development. Pull requests that require multiple rounds of revisions consume valuable engineering resources and slow development velocity. Identifying such high-friction pull requests before review begins could help teams allocate resources more effectively and reduce review overhead.

This project aims to build a machine learning model capable of predicting whether an AI-generated pull request will experience significant review friction. Specifically, we define friction as requiring two or more review cycles before approval. By detecting potentially problematic pull requests early, organizations can prioritize reviewer attention and improve software development efficiency.

The primary research question investigated is:

Predicting Friction in AI-Generated Code: Applying Machine Learning to Anticipate Heavy Review Cycles

Can machine learning accurately predict high-friction AI-generated pull requests using metadata available before the review process begins?

2. Background and Related Work

Machine learning has been widely applied in software analytics for defect prediction, effort estimation, code review analysis, and software quality assessment. Prior research has demonstrated that repository metadata and development metrics can be used to predict software engineering outcomes such as bug-prone files, code review delays, and maintenance complexity.

The emergence of AI-assisted software development introduces new challenges. Although AI coding assistants improve productivity, studies have shown that generated code may contain logical errors, security vulnerabilities, and maintainability concerns. Consequently, human reviewers often spend significant time validating AI-generated contributions.

Previous work in software analytics has focused on predicting code review outcomes using features such as code churn, file complexity, developer experience, and repository activity. However, limited research has explored the prediction of review friction specifically within AI-generated code contributions.

This project extends existing software analytics approaches by focusing on AI-generated pull requests and identifying the factors most strongly associated with costly review cycles.

3. Methodology

3.1 Dataset

The study utilizes the AIDev-pop dataset consisting of approximately 33,600 AI-generated pull requests. The dataset contains contributions generated by multiple AI coding agents, including:

- OpenAI Codex
- GitHub Copilot
- Cursor
- Devin

Predicting Friction in AI-Generated Code: Applying Machine Learning to Anticipate Heavy Review Cycles

- Claude Code

Each pull request includes metadata describing code changes, development activity, and review outcomes.

3.2 Defining Friction

A key aspect of the project involved defining what constitutes review friction.

- **0–1 Review Cycles:** Smooth merge process.
- **2 Review Cycles:** Friction event.
- **3+ Review Cycles:** Severe friction.

For the classification task, pull requests requiring two or more review cycles were labeled as the positive class.

The dataset exhibited significant class imbalance:

- Smooth Merges (0–1 cycles): 86%
- High-Friction Reviews (≥2 cycles): 14%

This imbalance created challenges for traditional machine learning models.

3.3 Feature Engineering

Several features were engineered from raw pull request metadata:

Code Complexity Features

- Total additions
- Total deletions
- Total changes
- Files touched

Timeline Features

- Hours to merge

Predicting Friction in AI-Generated Code: Applying Machine Learning to Anticipate Heavy Review Cycles

Context Features

- Pull request description length

Categorical Features

- AI agent identity
- Task type (Bug Fix, Feature Development, Documentation)

Categorical variables were transformed using one-hot encoding.

3.4 Model Development

Three classification algorithms were evaluated:

Logistic Regression

A linear baseline model used to establish initial performance.

Random Forest

An ensemble learning method capable of capturing nonlinear relationships.

XGBoost

A gradient boosting algorithm optimized through hyperparameter tuning.

To address class imbalance, the project evaluated:

1. SMOTE (Synthetic Minority Oversampling Technique)
2. Class weighting strategies

After experimentation, XGBoost's built-in scale_pos_weight parameter produced the strongest results while maintaining model stability.

Dimensionality reduction techniques such as Principal Component Analysis (PCA) were intentionally avoided to preserve feature interpretability.

Predicting Friction in AI-Generated Code: Applying Machine Learning to Anticipate Heavy Review Cycles

4. Results and Discussion

4.1 Model Performance

Performance was evaluated using a holdout test set representing 20% of the dataset.

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	75.3%	34.4%	84.5%	48.9%
Random Forest	89.5%	69.2%	45.1%	54.6%
XGBoost	85.6%	49.0%	80.7%	61.0%

The Random Forest achieved the highest overall accuracy but failed to identify a large proportion of high-friction pull requests. Logistic Regression achieved strong recall but produced excessive false positives.

XGBoost provided the best balance between recall and precision, making it the most practical model for deployment.

4.2 Feature Importance Analysis

Feature importance analysis revealed several notable findings:

Code Deletions Matter More Than Additions

Code deletions emerged as one of the strongest predictors of review friction. This suggests that reviewers are more cautious when AI systems remove or refactor existing code than when they introduce new code.

AI Agent Identity is Highly Predictive

Predicting Friction in AI-Generated Code: Applying Machine Learning to Anticipate Heavy Review Cycles

The specific AI coding assistant used was among the most influential features. Certain agents produced pull requests that were significantly more likely to trigger additional review cycles.

Task Type Has Limited Influence

Contrary to expectations, task categories such as bug fixes, features, and documentation updates contributed less predictive power than code modification characteristics and agent identity.

4.3 Practical Implications

The final model successfully identifies approximately 8 out of every 10 high-friction pull requests before extensive review effort is invested.

Potential applications include:

- Automated reviewer assignment
- Risk-based code review prioritization
- Early developer feedback systems
- Engineering productivity monitoring

These capabilities could significantly reduce review bottlenecks and improve team efficiency.

5. Threats to Validity

Several limitations should be considered when interpreting the results.

Internal Validity

The study relies on metadata-derived features and may not capture deeper semantic properties of source code. Important contextual information may therefore be omitted.

Construct Validity

Predicting Friction in AI-Generated Code: Applying Machine Learning to Anticipate Heavy Review Cycles

Review friction was measured using review cycle counts. Although practical, review cycles may not fully represent the complexity or quality of a pull request.

External Validity

The findings are based on the AIDev-pop dataset and may not generalize to all repositories, organizations, or development environments.

Dataset Bias

The distribution of AI agents and task types may influence model behavior. Future datasets with broader representation could improve generalizability.

6. Conclusion

This project demonstrates that machine learning can effectively predict high-friction AI-generated pull requests using metadata available before review begins. Among the evaluated approaches, XGBoost delivered the strongest overall performance, achieving 80.7% recall while maintaining reasonable precision.

The analysis revealed that code deletions and AI agent identity are stronger indicators of review friction than task type. These findings provide valuable insights into how organizations can better manage AI-assisted development workflows.

Future work may incorporate natural language processing on review comments, repository-specific models, and automated reviewer routing systems. As AI-generated code becomes increasingly common, predictive systems such as the one developed in this project can play an important role in preserving engineering productivity while maintaining software quality.

Predicting Friction in AI-Generated Code: Applying Machine Learning to Anticipate Heavy Review Cycles

References

[1] Breiman, L. "Random Forests." Machine Learning, 2001.

[2] Chen, T., and Guestrin, C. "XGBoost: A Scalable Tree Boosting System." KDD, 2016.

[3] Chawla, N. V., et al. "SMOTE: Synthetic Minority Over-sampling Technique." Journal of Artificial Intelligence Research, 2002.

[4] McIntosh, S., et al. "The Impact of Code Review Coverage and Code Review Participation on Software Quality." Empirical Software Engineering, 2016.

[5] GitHub Copilot Research and AI-Assisted Software Development Literature.

[6] AIDev-pop Dataset Documentation and Repository.